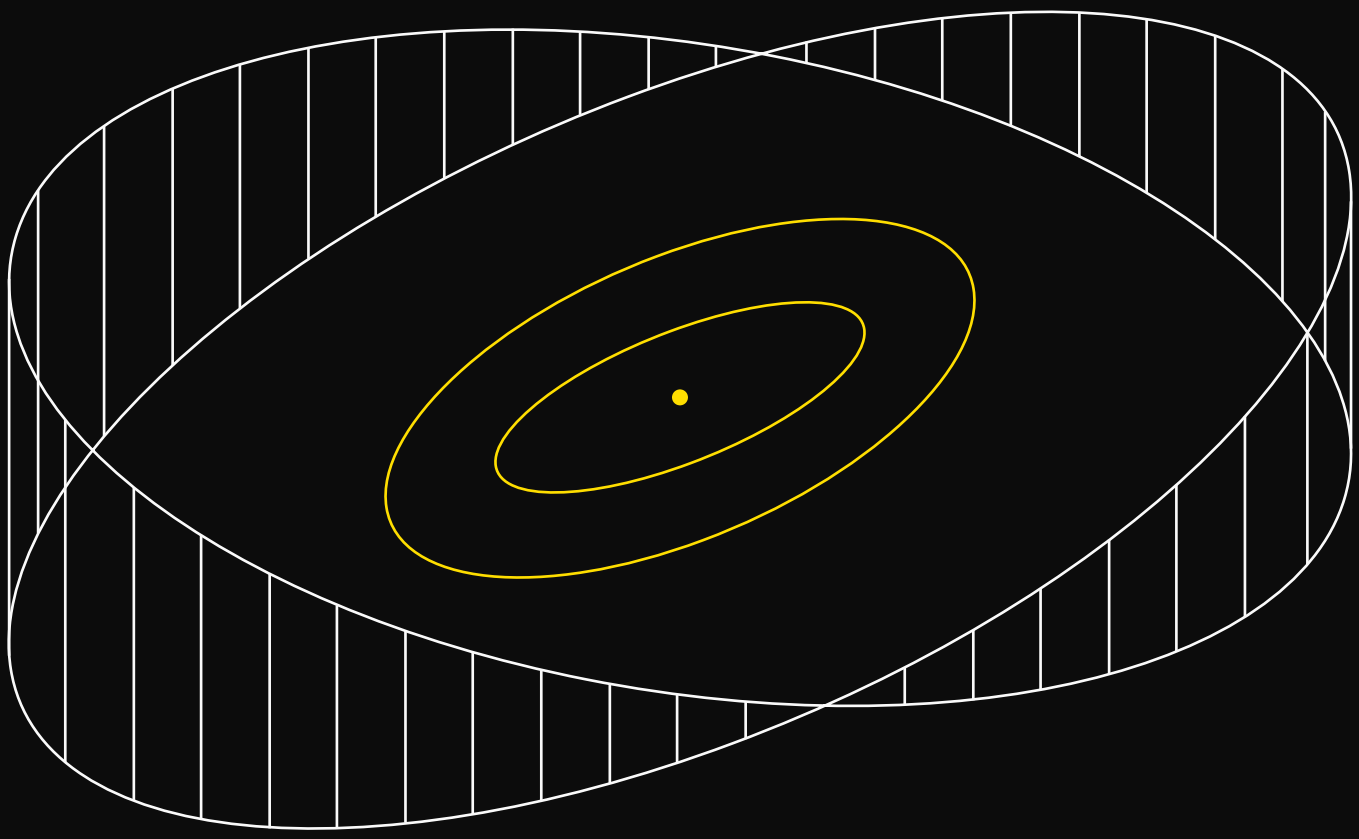
ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

Tool-Using RL envs test task



ИЗБРАННЫЕ ТЕМЫ ИССЛЕДОВАНИЙ В AI

ДОМАШНЕЕ ЗАДАНИЕ

НЕДЕЛЯ 4

Задача — сделать LLM-агента, который **взаимодействует со средой** в несколько шагов и использует тулы. Отличие от предыдущих домашних заданий в том, что сделать задачу полностью формализованной нельзя. Придётся придумывать прокси-награду, которая лучше всего оценит качество решения задачи.

СРЕДА

Сначала нужно придумать задачу агента. Это должна быть задача, которая удовлетворяет условиям:

→ У агента должно быть два типа действий:

1. **Free-text ответ** (сообщение пользователю/оператору/системе или другие тулы с free-text инпутом).
2. **Вызов тула** (строго структурированный) — они должны быть и на действие, и для получения какой-то информации о мире.

→ Сложность должна быть **регулируемой** (difficulty ∈ [1, 10]).

→ Датасет должен быть процедурно генерируемым, и должен существовать фиксированный eval.

Примеры миров (не обязательны)

- Дебаггер кода на Python.
- Планировщик встреч/ресурсов (работа с календарём).
- Агент, который проходит романтические новеллы.

ИНТЕРФЕЙСЫ

Data

Доработай `Data` из недели 1 под свои нужды. Например, если ты хочешь в нём хранить начальный стейт.

ToolEnv

Добавляем интерфейс многошаговой среды:

```
from abc import ABC, abstractmethod
from typing import Optional, Tuple, Dict, Any
from base.data import Data

class ToolEnv(ABC):
    """
    Multi-step tool-using environment.
    Text in / text out.
    """
    def __init__(self, name: str):
        self.name = name

    @abstractmethod
    def reset(self, data: Data) -> str:
        """
        Initialize an episode from Data and return initial
observation (text).
        """
        raise NotImplementedError

    @abstractmethod
    def step(self, action: str) -> Tuple[str, float, bool, Dict[str,
Any]]:
        """
        Apply one agent action and return:
          observation (text), reward (float), done (bool), info
(dict)
        """
        raise NotImplementedError

    @abstractmethod
    def generate(self, num_of_questions: int = 100, max_attempts: int
= 100,
                 difficulty: Optional[int] = 1, **kwargs) ->
list[Data]:
        """
        Procedurally generate episodes. Must support both difficulty
and direct hyperparams via kwargs.
        """
        raise NotImplementedError
```

Формат action (обязательный)

В каждом шаге агент возвращает строку, которая должна быть либо:

1. **Free-text сообщение**, например:

```
I can do that. Please confirm: move meeting M12 to Tue
15:00–15:30.
```

2. **Tool call** в строгом формате:

```
TOOL_CALL {"name": "get_calendar", "args": {"person": "P3",
"range": [0, 2]}}
```

`generate`

Требования как в неделе 1:

- **`num_of_questions`** — сколько эпизодов сгенерировать.
- **`max_attempts`** — сколько попыток на генерацию одного валидного эпизода.
- **`difficulty`** — целое от 1 до 10.
- **`*kwargs`** — возможность задавать гиперпараметры напрямую (не только difficulty).

ВАЖНО

1. Тебе нужно уметь генерировать фиксированные eval-наборы: например, **`eval_d1.jsonl`**, **`eval_d2.jsonl`**, ..., **`eval_d5.jsonl`** — по 100 эпизодов каждый.
2. Сложность должна реально расти (больше ограничений, больше длина эпизода и так далее).

`verifier`/МЕТРИКИ/REWARD

Теперь верификация — это уже не сравнение с голдом, а набор наград, которые ты самостоятельно задизайнишь, для последовательности действий агента в этой среде.

Сделай **`Verifier`**, который проверяет эпизод:

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any, Tuple, Optional
from base.data import Data

class TrajectoryVerifier(ABC):
    """
    Verifier that evaluates a *given* action trajectory in a multi-
step ToolEnv.
    It does NOT call the agent and does NOT generate actions.
    """

    @abstractmethod
    def verify_trajectory(
        self,
        env,
        data: Data,
        actions: List[str],
        max_steps: Optional[int] = None,
    ) -> Dict[str, Any]:
        """
        Runs env on the provided actions and returns metrics.

        Required keys in the returned dict:
          - success: bool
          - total_reward: float
          - steps: int
          - tool_calls: int
          - policy_violations: int

        Recommended keys:
          - terminated_early: bool
          - invalid_actions: int
          - info_trace: List[dict]  (per-step env info)
        """
        raise NotImplementedError
```

Метрики

- **`success`** — задача решена корректно (или корректно распознано UNSAT, если ты поддерживаешь UNSAT).
- **`policy_violations`** — нарушения твоих правил (см. ниже).
- **`steps`** — число шагов.
- **`tool_calls`** — число вызовов tools.
- **`total_reward`** — сумма reward за эпизод (твой shaping + outcome).

“policy rules”

Ты **обязан** ввести как минимум два правила, которые проверяются автоматически:

1. **Confirmation rule**: любые действия, меняющие мир (например, **`create/move/cancel/reserve/update`**) разрешены только после явного подтверждения в free-text.
2. **No hallucinated entities**: нельзя использовать **`id`**/имена, которых не было в observation/tool outputs/initial_state.

ПОДСКАЗКА

Ты можешь добавить свои: лимит вопросов, запрет на лишние инструменты и так далее.)

Reward

Сделай reward вида:

- outcome-centric: **`+1`** за success, **`1`** за fail;
- shaping: штрафы за **`tool_calls`**, **`steps`**, **`policy_violations`**, невалидный формат **`action`**, зацикливания агента и так далее.

Задача 1. Среда + агент + сравнение двух политик

1. Придумай задачу, дизайн среды и тулов.
2. Определи правила в твоей среде (policy constraints):
   минимум:
   a) изменяющие стейт тулы вызываются только после явного подтверждения пользователя;
   b) запрет на «галлюцинацию» сущностей в тулах (аргументы тулов должны быть корректными).
3. Реализуй среду **`ToolEnv`**.
4. Сгенерируй train/eval-датасеты.
5. Реализуй верификатор траектории **`verify_trajectory(env, data, actions, ...) -> metrics`**.
6. Собери бейзлайн на промптинге.
7. Обучи GRPO в твоей среде.
8. Сравни бейзлайн с обученной моделью и проанализируй результаты на всех eval-корзинках.
9. Опиши примеры, в которых оптимизация твоей прокси-награды может плохо влиять на реальное качество агента и предложи варианты решения этих проблем.

Задача 2*. Обучение агента на нескольких средах

Объедини две среды, которые у тебя получились на неделе 1 и в первой части этой домашней работы. Попробуй обучить политику сразу под две эти задачи.

ПРАВИЛА

- Проанализируй полученные результаты. Это **самый важный пункт**, потому что хочется увидеть не только числа с полученными метриками. Как ты объяснишь увиденное поведение? Напиши **отчёт** о проведённых экспериментах. Что получилось? Что нет?
- **Нет правильного способа решить задачу.** Не стоит беспокоиться, что ты делаешь что-то неправильно. Мы хотим увидеть твои способности к исследованиям, а не какое-то конкретное решение задачи. Возможно, ты придумаешь то, о чём мы даже не задумывались изначально, — это будет высший класс.
- Следуй интерфейсам, описанным в условиях задачи.
- Ты можешь использовать Google Colab или Kaggle Code, чтобы получить доступ к бесплатным вычислительным ресурсам.
- Присылай решение в виде репозитория на GitHub с отчётом по решению и чёткими инструкциями, как запустить твой код. **Убедись, что мы сможем запустить решение по этим инструкциям.**
- Ты можешь использовать любые библиотеки и фреймворки, которые могут быть необходимы.
- **Сфокусируйся на том, чтобы код был чистым и понятным.** Если считаешь, что какая-то его часть может быть непонятна, то добавь комментарии. Мы очень ценим хорошо написанный код, поэтому **если решение задачи будет оформлено грязно, то можем его отклонить**.